

# International Institute of Information Technology, Bangalore



International  
Institute of Information  
Technology Bangalore

**CSE 816: Software Production Engineering**

---

## Final Project

**Automated MLOps Pipeline with Drift Detection**

---

**Student Name**

Athira Lonappan  
Kirti Nigam

**Student ID**

MT2025027  
MT2025066

**Lecturer in charge:**

Prof. Thangaraju B

**Submission Date : 10th May 2026**

## Abstract

This report describes a software programming engineering project implementing an end-to-end **drift-aware customer churn prediction** system for a banking-style use case. Four **Flask** microservices handle data ingestion, model training, online prediction, and statistical drift detection against a reference distribution saved at training time. The platform runs in **Docker** and **Kubernetes**, with a shared persistent volume for model artifacts, **horizontal pod autoscaling**, **Jenkins** continuous integration and deployment, **Ansible** provisioning with **Vault**-encrypted secrets, and an **ELK** stack (**Elasticsearch**, **Filebeat**, **Kibana**) for structured logs and dashboards. When drift is detected, the drift service can optionally trigger retraining by calling the training API. The report summarizes requirements, machine learning design, service APIs, deployment topology, CI/CD, observability, security considerations, and evaluation scenarios.

# Contents

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                        | <b>1</b>  |
| 1.1      | Background and Motivation . . . . .        | 1         |
| 1.2      | Problem Statement . . . . .                | 1         |
| 1.3      | Project Goals . . . . .                    | 1         |
| 1.4      | Scope and Assumptions . . . . .            | 2         |
| 1.5      | Project Structure . . . . .                | 2         |
| <b>2</b> | <b>Requirements and System Overview</b>    | <b>3</b>  |
| 2.1      | Functional Requirements . . . . .          | 3         |
| 2.2      | Non-Functional Requirements . . . . .      | 3         |
| 2.3      | High-Level Architecture . . . . .          | 4         |
| 2.4      | Technology Stack . . . . .                 | 4         |
| <b>3</b> | <b>Machine Learning Design</b>             | <b>5</b>  |
| 3.1      | Dataset and Features . . . . .             | 5         |
| 3.2      | Model . . . . .                            | 5         |
| 3.3      | Reference Distribution . . . . .           | 5         |
| 3.4      | Drift Detection Method . . . . .           | 6         |
| 3.5      | Retraining Policy . . . . .                | 6         |
| <b>4</b> | <b>Microservices and APIs</b>              | <b>7</b>  |
| 4.1      | Data Ingestion Service . . . . .           | 7         |
| 4.2      | Model Training Service . . . . .           | 7         |
| 4.3      | Model Serving Service . . . . .            | 8         |
| 4.4      | Drift Detection Service . . . . .          | 8         |
| 4.5      | Inter-Service Communication . . . . .      | 9         |
| <b>5</b> | <b>Platform and Deployment</b>             | <b>10</b> |
| 5.1      | Containerization . . . . .                 | 10        |
| 5.2      | Kubernetes Resources . . . . .             | 10        |
| 5.3      | Networking and Service Discovery . . . . . | 11        |
| 5.4      | Configuration and Secrets . . . . .        | 11        |
| 5.5      | Infrastructure as Code . . . . .           | 12        |
| <b>6</b> | <b>CI/CD and Quality Assurance</b>         | <b>13</b> |
| 6.1      | Environment Setup . . . . .                | 13        |
| 6.2      | Run Unit Tests . . . . .                   | 13        |
| 6.3      | Integration Testing . . . . .              | 13        |
| 6.4      | Docker Login . . . . .                     | 13        |
| 6.5      | Build & Push Images . . . . .              | 14        |
| 6.6      | Deploy to Kubernetes . . . . .             | 14        |

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| 6.7       | ELK Dashboard Setup . . . . .                        | 14        |
| <b>7</b>  | <b>Observability and Operations</b>                  | <b>15</b> |
| 7.1       | Structured Logging . . . . .                         | 15        |
| 7.2       | ELK Stack . . . . .                                  | 15        |
| 7.3       | Dedicated Dashboards . . . . .                       | 16        |
| 7.4       | Monitoring and Troubleshooting . . . . .             | 17        |
| 7.5       | Operational Scenarios . . . . .                      | 17        |
| <b>8</b>  | <b>Root Cause Analysis (RCA) and XAI Diagnostics</b> | <b>18</b> |
| 8.1       | Overview: What is happening? . . . . .               | 18        |
| 8.2       | Motivation: Why are we implementing it? . . . . .    | 18        |
| 8.3       | The XAI Diagnostic Dashboard . . . . .               | 18        |
| 8.3.1     | Dashboard Layout and Navigation . . . . .            | 19        |
| 8.3.2     | Graphs and Visualizations . . . . .                  | 19        |
| 8.3.3     | Interactive Filters . . . . .                        | 20        |
| 8.4       | Innovation: Why is this innovative? . . . . .        | 20        |
| <b>9</b>  | <b>Security and Governance</b>                       | <b>21</b> |
| 9.1       | Secrets Management . . . . .                         | 21        |
| 9.2       | Network and Access . . . . .                         | 21        |
| 9.3       | Data and Model Lifecycle . . . . .                   | 21        |
| <b>10</b> | <b>Results and Evaluation</b>                        | <b>22</b> |
| 10.1      | Functional Validation . . . . .                      | 22        |
| 10.2      | End-to-End Behavior . . . . .                        | 22        |
| 10.3      | Limitations . . . . .                                | 22        |
| <b>11</b> | <b>Visual Appendix</b>                               | <b>24</b> |
| 11.1      | Pipeline Execution . . . . .                         | 24        |
| 11.2      | Pod Logs . . . . .                                   | 24        |
| 11.3      | Kibana Dashboards . . . . .                          | 25        |
| 11.4      | Operational Scenarios . . . . .                      | 27        |
| <b>12</b> | <b>Conclusion and Future Work</b>                    | <b>28</b> |
| 12.1      | Summary . . . . .                                    | 28        |
| 12.2      | Lessons Learned . . . . .                            | 28        |
| 12.3      | Future Improvements . . . . .                        | 29        |
| <b>13</b> | <b>References</b>                                    | <b>30</b> |

# Chapter 1: Introduction

## 1.1 Background and Motivation

Customer churn prediction has become a critical challenge in the banking sector, where retaining existing customers is often more cost-effective than acquiring new ones. Banks deal with large volumes of tabular customer data, including demographics, transaction histories, and product usage patterns. Predictive models can score churn risk and help institutions proactively intervene. However, deploying such models in production requires more than just training accuracy — it demands operational discipline. Without robust MLOps practices, models risk becoming stale, unmonitored, and misaligned with evolving customer behavior. This project is motivated by the need to bridge the gap between machine learning research and production-grade systems, ensuring that churn prediction pipelines remain reliable, scalable, and trustworthy.

## 1.2 Problem Statement

The central problem addressed in this project is the scoring of churn risk from tabular customer data in a production environment. While training a model is straightforward, maintaining its relevance over time is more complex. Customer behavior changes, regulatory requirements evolve, and new data distributions emerge. These shifts introduce *data drift*, which can degrade model performance and erode trust in predictions. The challenge is therefore twofold: (1) build a system that can deliver churn risk scores consistently, and (2) detect and respond to drift so that models remain valid and actionable in real-world banking operations.

## 1.3 Project Goals

This project aims to design and implement an MLOps pipeline with the following goals:

- **Microservice architecture:** Modular services for data ingestion, preprocessing, model scoring, and drift detection, enabling independent scaling and maintainability.
- **Automated CI/CD:** Continuous integration and deployment pipelines to ensure that code, models, and configurations are tested and deployed seamlessly.
- **Observability:** Integration of logging and monitoring (via the ELK stack) to provide visibility into pipeline health, performance, and anomalies.

- **Optional retraining:** Trigger retraining workflows when drift is detected, ensuring that models adapt to new data distributions without manual intervention.

Together, these goals align with the broader vision of production-ready machine learning systems that are resilient, transparent, and adaptive.

## 1.4 Scope and Assumptions

The scope of this project is deliberately bounded to ensure feasibility within the academic context:

- **In scope:** Four microservices (data ingestion, preprocessing, model scoring, drift detection), Kubernetes-based deployment, CI/CD automation, and ELK stack integration for observability.
- **Out of scope:** Advanced hyperparameter optimization, multi-cloud deployment, federated learning, and large-scale distributed training. These are acknowledged as potential extensions but not implemented here.
- **Assumptions:** The system assumes access to structured tabular customer data, a Kubernetes cluster for deployment, and sufficient resources to run Elasticsearch and Kibana for monitoring. It also assumes that retraining can be triggered based on drift signals, though retraining frequency and governance policies are simplified for this project.

## 1.5 Project Structure

```

SPE_FINAL_PROJECT/
├─ data_ingestion/           # Ingest API + tests
├─ model_training/           # Train API, health, tests
├─ model_serving/            # Predict API + tests
├─ drift_detection/          # Drift API + tests
├─ kubernetes/
│   ├─ deployment/           # Per-microservice Deployments
│   └─ service/              # ClusterIP Services
│       ├─ pv.yaml, pvc.yaml
│       └─ hpa.yaml
│   └─ elk/                  # Elasticsearch, Kibana, Filebeat, dashboards, setup Job
├─ ansible/                  # Roles, inventory, Vault secrets
├─ tests/
│   └─ test_integration.py    # pytest-style pipeline checks (optional locally)
├─ docker-compose.test.yml   # Local stack for Jenkins / manual testing
├─ requirements.txt           # Python deps for tests / tooling
└─ Jenkinsfile

```

# Chapter 2: Requirements and System Overview

## 2.1 Functional Requirements

The system must support the following core functionalities:

- **Data Ingestion:** Collect tabular customer data from banking systems, ensuring schema consistency and preprocessing readiness.
- **Model Training:** Train predictive models using scikit-learn, with modular pipelines for reproducibility.
- **Prediction Service:** Provide churn risk scores via a RESTful API, enabling integration with customer-facing applications.
- **Drift Detection:** Continuously monitor input data distributions against reference datasets to detect drift.
- **Optional Retraining:** When drift is detected, trigger retraining workflows via a configurable `TRAINING_URL`, ensuring models remain trustworthy.

| Service         | Port (default) | Role                                                                                                                                                                                               |
|-----------------|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data ingestion  | 5000           | <code>POST /ingest</code> — validate payload, call serving + drift; response includes <code>drift_response</code> (with <code>drift_detected</code> ). Logs as <code>service.name: ingest</code> . |
| Model training  | 5001           | <code>POST /train</code> — fit model + reference; <code>GET /health</code> for compose healthchecks.                                                                                               |
| Model serving   | 5002           | <code>POST /predict</code> — load model from PVC, return predictions.                                                                                                                              |
| Drift detection | 5003           | <code>POST /drift</code> — returns <code>drift_detected</code> + <code>details</code> ; calls <code>TRAINING_URL</code> (retrain) when drift is detected and that env var is set.                  |

Figure 2.1: Functional requirements summary.

## 2.2 Non-Functional Requirements

Beyond core functionality, the system must adhere to production-grade qualities:

- **Scalability:** Horizontal Pod Autoscaler (HPA) ensures services scale based on workload.

- **Health and Readiness Probes:** Kubernetes probes guarantee that only healthy services receive traffic.
- **Structured Logging:** Logs are standardized and shipped to the ELK stack for observability.
- **Secrets Handling:** Sensitive information (API keys, database credentials) is managed securely via Kubernetes Secrets.

| Non-Functional Requirement | Implementation                       |
|----------------------------|--------------------------------------|
| Scalability                | Horizontal Pod Autoscaler (HPA)      |
| Health Checks              | Kubernetes liveness/readiness probes |
| Logging                    | Structured logs shipped to ELK stack |
| Secrets                    | Managed via Kubernetes Secrets       |

Table 2.1: Non-functional requirements summary.

## 2.3 High-Level Architecture

The system architecture follows a modular microservice design:

- **Data Flow:** Customer data is ingested, processed, and passed to the prediction service.
- **Shared PVC:** A Persistent Volume Claim stores the trained model and reference dataset, accessible by both prediction and drift detection services.
- **Drift Detection:** When drift is detected, alerts are raised and retraining may be triggered via the `TRAINING_URL`.
- **Serving Layer:** Predictions and drift signals are exposed via REST APIs.

## 2.4 Technology Stack

| Layer                  | Technology                                                                                                                                                  |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| APIs                   | Python Flask                                                                                                                                                |
| ML                     | pandas, numpy, scipy, scikit-learn                                                                                                                          |
| Containers             | Docker                                                                                                                                                      |
| Orchestration          | Kubernetes (Deployments with <b>rolling updates</b> + <code>/health</code> readiness, Services, PV/PVC, HPA on ingestion, serving, drift — not on training) |
| CI/CD                  | Jenkins ( <code>Jenkinsfile</code> : Ansible, unit tests, compose integration, image build/push, K8s deploy, ELK setup)                                     |
| Infrastructure as code | Ansible ( <code>ansible/site.yml</code> , Vault-encrypted secrets)                                                                                          |
| Logs / dashboards      | Elasticsearch 7.17, Filebeat (JSON container logs → <code>project-logs-*</code> ), Kibana (imported dashboards)                                             |
| Security               | Ansible Vault (AES-256) for sensitive variables                                                                                                             |

Figure 2.2: Technology Stack Summary.



# Chapter 3: Machine Learning Design

## 3.1 Dataset and Features

The dataset used for churn prediction consists of tabular customer records from the banking domain. Each record includes demographic, financial, and behavioral attributes. The target variable is **Exited**, indicating whether a customer has churned.

| Feature         | Description                              |
|-----------------|------------------------------------------|
| CreditScore     | Customer credit score                    |
| Geography       | Country/region of the customer           |
| Gender          | Male/Female                              |
| Age             | Customer age                             |
| Tenure          | Years of relationship with the bank      |
| Balance         | Account balance                          |
| NumOfProducts   | Number of products held                  |
| HasCrCard       | Whether customer has a credit card       |
| IsActiveMember  | Activity status of the customer          |
| EstimatedSalary | Estimated annual salary                  |
| Exited          | Target label (1 = churned, 0 = retained) |

Table 3.1: Churn dataset schema and features.

## 3.2 Model

The model design follows a preprocessing + logistic regression pipeline:

- **Preprocessing:** Encoding categorical variables (e.g., Geography, Gender), scaling numerical features (e.g., CreditScore, Age).
- **Logistic Regression:** A baseline classifier chosen for interpretability and efficiency.
- **Artifacts:** The trained model is serialized as `churn_model.pkl` for deployment.

## 3.3 Reference Distribution

To ensure robustness at inference time, the system maintains a reference distribution of features:

- Stored as `reference_distribution.pkl`.
- Captures baseline feature distributions from training data.
- Used to compare incoming data distributions during prediction.

### 3.4 Drift Detection Method

Drift detection is performed using statistical tests:

- **Per-feature drift:** Kolmogorov–Smirnov (KS) style tests compare incoming feature distributions against the reference.
- **Label drift:** Monitoring the distribution of the target variable `Exited`.
- **Outputs:** A boolean flag `drift_detected` and detailed metrics per feature.

| Drift Type    | Detection Method                               |
|---------------|------------------------------------------------|
| Feature Drift | KS test per feature vs. reference              |
| Label Drift   | Distribution monitoring on <code>Exited</code> |
| Output        | <code>drift_detected</code> + drift details    |

Table 3.2: Drift detection methods and outputs.

### 3.5 Retraining Policy

Retraining is governed by the following policy:

- Triggered when `drift_detected` is true.
- Requires `TRAINING_URL` to be configured.
- Retraining is coupled to the ingestion path, ensuring updated models are trained on fresh data.
- Retrained models replace the existing `churn_model.pkl` artifact.

# Chapter 4: Microservices and APIs

## 4.1 Data Ingestion Service

The data ingestion service is responsible for receiving customer data and orchestrating downstream services.

- Endpoint: POST /ingest
- Responsibilities: Schema validation, persistence, and triggering serving and drift detection services.
- Ensures that only valid tabular data enters the pipeline.
- Hit the command **kubectrl port-forward deployment/data-ingestion 5000:5000**

| Endpoint     | Function                                                          |
|--------------|-------------------------------------------------------------------|
| POST /ingest | Accepts customer data, validates schema, triggers serving + drift |

Table 4.1: Data ingestion service API summary.

```
kj@LAPTOP-8B10SK0:~/SPE_FINAL_PROJECT$ curl -s -X POST http://localhost:5000/ingest -H 'Content-Type: application/json' -d '{
  "CustomerId": 123,
  "CreditScore": 700,
  "Geography": "Spain",
  "Gender": "Female",
  "Age": 35,
  "Tenure": 3,
  "Balance": 2500.0,
  "NumOfProducts": 1,
  "HasCrCard": 1,
  "IsActiveMember": 1,
  "EstimatedSalary": 45000.0,
  "Exited": 0
},
{
  "CustomerId": 1234,
  "CreditScore": 600,
  "Geography": "Germany",
  "Gender": "Female",
  "Age": 35,
  "Tenure": 4,
  "Balance": 4000.0,
  "NumOfProducts": 1,
  "HasCrCard": 1,
  "IsActiveMember": 1,
  "EstimatedSalary": 60000.0,
  "Exited": 1
}'
{"drift_response":{"details":{"Age_drift":false,"Balance_drift":false,"CreditScore_drift":false,"CustomerId_drift":false,"EstimatedSalary_drift":false,"HasCrCard_drift":false,"IsActiveMember_drift":false,"NumOfProducts_drift":false,"Tenure_drift":false,"concept_drift":{"monitor_prediction_accuracy_over_time":{"label_drift":true},"drift_detected":true,"training_response":{"message":"Model and reference saved to PVC","status":"success"},"rows":2,"serving_response":{"results":[{"CustomerId":123,"churn_probability":0.9949134411077678,"prediction":1}]},"CustomerId":1234,"churn_probability":0.9984707442492561,"prediction":1},"status":"success"},"status":"ingested"}}
```

Figure 4.1: Workflow of the data ingestion service.

## 4.2 Model Training Service

The training service manages model retraining and health monitoring.

- Endpoints: POST /train, GET /health
- Supports training from PVC-stored datasets or inline JSON payloads.

- Produces structured logs for reproducibility and observability.
- Hit the command **kubectrl port-forward deployment/model-training 5001:5001**

| Endpoint    | Function                                      |
|-------------|-----------------------------------------------|
| POST /train | Initiates model training (PVC or inline JSON) |
| GET /health | Returns service health status                 |

Table 4.2: Model training service API summary.

```
kirti@LAPTOP-8B10T5K0:~/SPE/SPE_FINAL_PROJECT$ curl -X POST http://localhost:5001/train
{"message":"Model and reference saved to PVC","status":"success"}
```

Figure 4.2: Model training workflow and artifact generation.

## 4.3 Model Serving Service

The serving service provides churn risk predictions.

- Endpoint: POST /predict
- Accepts feature schema identical to training data.
- Ignores the target label **Exited** if present in payload.
- Hit the command **kubectrl port-forward deployment/model-serving 5002:5002**

| Endpoint      | Function                                    |
|---------------|---------------------------------------------|
| POST /predict | Returns churn risk score for input features |

Table 4.3: Model serving service API summary.

```
kirti@LAPTOP-8B10T5K0:~/SPE/SPE_FINAL_PROJECT$ curl -X POST http://localhost:5002/predict -H "Content-Type: application/json" -d '{"Customer
Id":123,"CreditScore":700,"Geography":"Spain","Gender":"Female","Age":35,"Tenure":3,"Balance":2500.0,"NumOfProducts":1,"HasCrCard":1,"IsActiveMe
mber":1,"EstimatedSalary":45000.0,"Exited":0}'
{"results":[{"CustomerId":123,"churn_probability":0.9949134411077676,"prediction":1}], "status":"success"}
```

Figure 4.3: Prediction workflow in the serving service.

## 4.4 Drift Detection Service

The drift detection service monitors data distributions and signals retraining when necessary.

- Endpoint: POST /drift
- Response includes drift flag and per-feature details.
- Optional field: **training\_response** if retraining is triggered.
- Hit the command **kubectrl port-forward deployment/drift-detection 5003:5003**

| Endpoint    | Function                                                      |
|-------------|---------------------------------------------------------------|
| POST /drift | Detects drift, returns flag + details, may trigger retraining |

Table 4.4: Drift detection service API summary.

```

k1t1@LAPTOP-BB10T5K0:~/SPE/SPE_FINAL_PROJECT$ curl -X POST http://localhost:5003/drift \
-H "Content-Type: application/json" \
-d '{"CustomerId":15749177,"CreditScore":627,"Geography":"France","Gender":"Male","Age":33,"Tenure":1,"Balance":0.0,"NumOfProducts":2,"HasCrCard":1,"IsActiveMember":1,"EstimatedSalary":49503.5}{'
{"details":{"Age_drift":false,"Balance_drift":false,"CreditScore_drift":false,"CustomerId_drift":false,"EstimatedSalary_drift":false,"HasCrCard_drift":false,"IsActiveMember_drift":false,"NumOfProducts_drift":false,"Tenure_drift":false,"concept_drift":"monitor prediction accuracy over time"},"drift_detected":false}

```

Figure 4.4: Drift detection workflow and retraining trigger.

## 4.5 Inter-Service Communication

Microservices communicate via environment variables and standardized request/response contracts:

- Environment variables: `SERVING_URL`, `DRIFT_URL`, `TRAINING_URL`.
- Contracts: JSON-based requests and responses with schema validation.
- Ensures modularity and loose coupling between services.

| Variable                  | Purpose                              |
|---------------------------|--------------------------------------|
| <code>SERVING_URL</code>  | Endpoint for prediction service      |
| <code>DRIFT_URL</code>    | Endpoint for drift detection service |
| <code>TRAINING_URL</code> | Endpoint for retraining service      |

Table 4.5: Environment variables for inter-service communication.

# Chapter 5: Platform and Deployment

## 5.1 Containerization

Each microservice is packaged as a Docker image, ensuring portability and reproducibility across different environments. Containerization allows developers to encapsulate dependencies, configurations, and runtime environments, thereby eliminating the common issue of "works on my machine". This approach ensures that the same image can be deployed consistently in development, testing, and production clusters. Additionally, Docker images provide versioning capabilities, enabling rollback to previous stable builds when necessary.

- Data ingestion: Handles schema validation and orchestration, ensuring that raw customer data is properly formatted before entering the pipeline.
- Model training: Builds and updates churn models, encapsulating preprocessing steps and producing artifacts such as `churn_model.pkl`.
- Model serving: Provides predictions via REST API, exposing endpoints that integrate seamlessly with external applications.
- Drift detection: Monitors data distributions and signals retraining, ensuring that models remain trustworthy over time.

| Service         | Docker Responsibility                        |
|-----------------|----------------------------------------------|
| Ingestion       | Validate schema, trigger downstream services |
| Training        | Train model, produce artifacts               |
| Serving         | Serve predictions via API                    |
| Drift Detection | Monitor drift, trigger retraining            |

Table 5.1: Docker images and responsibilities per service.

## 5.2 Kubernetes Resources

Deployment is orchestrated via Kubernetes, which provides scalability, resilience, and automated management of workloads. Each microservice is defined as a Deployment, ensuring that pods are recreated in case of failure. Services expose these deployments internally, enabling reliable communication between ingestion, training, serving, and drift detection components. Persistent Volume Claims (PVCs) are used to store model artifacts, ensuring that retrained models

are consistently available across pods. Horizontal Pod Autoscalers (HPAs) dynamically adjust the number of replicas based on CPU or memory usage, ensuring efficient resource utilization.

- Deployments and Services for each microservice, ensuring modularity and independent scaling.
- Persistent Volume Claim (PVC) for `/data/churn-model`, maintaining consistency of model artifacts.
- Horizontal Pod Autoscaler (HPA) for scaling serving workloads, ensuring responsiveness under varying loads.

## 5.3 Networking and Service Discovery

Kubernetes networking ensures that microservices can discover and communicate with each other reliably. ClusterIP services expose endpoints internally, avoiding unnecessary external exposure. Each service is mapped to a specific port, ensuring clear separation of responsibilities. Service discovery is handled by Kubernetes DNS, allowing microservices to reference each other by name rather than IP addresses. This abstraction simplifies configuration and improves maintainability.

- Services exposed via ClusterIP, ensuring internal-only communication.
- Ports: 5000–5003 mapped to ingestion, training, serving, and drift detection.
- DNS-based service discovery, enabling microservices to communicate using logical names.

## 5.4 Configuration and Secrets

Configuration management is handled through environment variables, which allow flexible adaptation across environments without modifying code. Sensitive information such as database credentials or API keys is managed securely using Ansible Vault, ensuring encryption and controlled access. This approach enforces separation of concerns, where developers can focus on application logic while operators manage secure configurations.

- Environment-based configuration for service URLs, ensuring portability across environments.
- Sensitive variables managed via Ansible Vault, providing encryption and controlled access.
- Separation of configuration from code, improving maintainability and security.

## 5.5 Infrastructure as Code

Infrastructure provisioning and deployment are automated using Ansible, which provides declarative playbooks and reusable roles. This ensures consistency across environments and reduces manual intervention. Roles encapsulate common tasks such as Kubernetes setup, ELK stack deployment, and secrets management. Playbooks orchestrate these roles, enabling end-to-end automation from cluster setup to application deployment. This Infrastructure as Code (IaC) approach improves reproducibility, reduces errors, and accelerates development cycles.

- Ansible roles for Kubernetes and ELK stack, encapsulating reusable deployment logic.
- Playbooks automate deployment and configuration, ensuring consistent environments.
- Infrastructure as Code improves reproducibility and reduces manual errors.



# Chapter 6: CI/CD and Quality Assurance

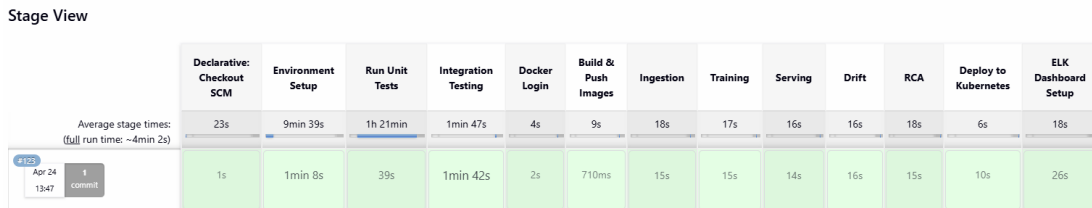


Figure 6.1: Jenkins pipeline stages for CI/CD and quality assurance.

## 6.1 Environment Setup

This stage prepares the deployment environment by applying Ansible playbooks with vault-protected secrets. Credentials are securely injected, and configuration files are provisioned to ensure that Kubernetes manifests, ELK stack components, and microservice dependencies are consistently deployed across environments.

## 6.2 Run Unit Tests

Unit tests validate the correctness of individual microservices before integration. Each service (ingestion, drift detection, serving, training) is tested in isolation using Python's `unittest` framework. This ensures schema validation, prediction logic, and drift detection functions behave as expected, reducing the risk of runtime errors in production.

## 6.3 Integration Testing

Integration testing spins up all services via Docker Compose and validates end-to-end workflows. The pipeline trains a baseline model, ingests sample customer data, and checks prediction and drift detection responses. Timeouts and error handling are enforced, with logs collected for diagnostics. This stage ensures that services communicate correctly and artifacts are generated reliably.

## 6.4 Docker Login

This stage authenticates with Docker Hub using stored credentials. It ensures secure access to the container registry, enabling subsequent build and push oper-

ations. Credential management prevents exposure of sensitive information while maintaining automation.

## 6.5 Build & Push Images

Each microservice is containerized and pushed to Docker Hub in parallel:

- **Ingestion:** Builds and pushes the ingestion service image.
- **Training:** Builds and pushes the training service image.
- **Serving:** Builds and pushes the serving service image.
- **Drift Detection:** Builds and pushes the drift detection service image.

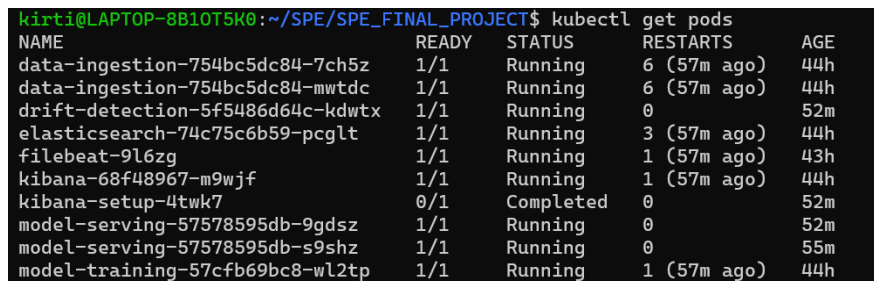
Parallelization reduces pipeline runtime and ensures all services are updated simultaneously.

## 6.6 Deploy to Kubernetes

Deployment manifests are applied to the Kubernetes cluster. In this project, the cluster was provisioned using **Minikube**, a lightweight Kubernetes distribution ideal for local development and testing. Minikube provided a single-node environment with limited resources, which was sufficient for deploying the four microservices and observability stack. Persistent Volumes and Claims were created for model storage, Deployments and Services were applied for each microservice, and Horizontal Pod Autoscalers were configured for scaling. This stage ensured reproducible, cluster-native deployment with automated scaling and resilience.

## 6.7 ELK Dashboard Setup

The final stage provisions observability infrastructure. Elasticsearch, Filebeat, and Kibana are deployed, with index mappings and dashboards configured. A dedicated `kibana-setup` job ensures dashboards are initialized correctly. Logs and rollout statuses are collected to verify successful setup, enabling structured monitoring and troubleshooting.



| NAME                             | READY | STATUS    | RESTARTS    | AGE |
|----------------------------------|-------|-----------|-------------|-----|
| data-ingestion-754bc5dc84-7ch5z  | 1/1   | Running   | 6 (57m ago) | 44h |
| data-ingestion-754bc5dc84-mwtde  | 1/1   | Running   | 6 (57m ago) | 44h |
| drift-detection-5f5486d64c-kdwtx | 1/1   | Running   | 0           | 52m |
| elasticsearch-74c75c6b59-pcglr   | 1/1   | Running   | 3 (57m ago) | 44h |
| filebeat-9l6zg                   | 1/1   | Running   | 1 (57m ago) | 43h |
| kibana-68f48967-m9wjf            | 1/1   | Running   | 1 (57m ago) | 44h |
| kibana-setup-4twk7               | 0/1   | Completed | 0           | 52m |
| model-serving-57578595db-9gdsz   | 1/1   | Running   | 0           | 52m |
| model-serving-57578595db-s9shz   | 1/1   | Running   | 0           | 55m |
| model-training-57c6b69bc8-wl2tp  | 1/1   | Running   | 1 (57m ago) | 44h |

Figure 6.2: Kubernetes Pods after jenkins pipeline is run.

# Chapter 7: Observability and Operations

## 7.1 Structured Logging

Structured logging was implemented across all microservices to ensure consistency and machine-readability. Logs are emitted in JSON format with standardized fields:

- **service.name** — uniquely identifies the microservice (ingestion, training, serving, drift).
- **event** — describes the action (e.g., request received, drift detected, training completed).
- **timestamp** — ensures chronological ordering for troubleshooting and correlation.

This schema makes logs easy to parse and index in Elasticsearch. Training events, ingestion requests, and drift detection outcomes are logged for reproducibility. Pod logs collected via `kubectl logs` provide raw diagnostic data, which is then shipped into the ELK pipeline.

## 7.2 ELK Stack

The ELK stack forms the backbone of observability:

- **Filebeat**: Collects logs directly from Kubernetes pods and forwards them to Elasticsearch.
- **Elasticsearch**: Stores logs under the index pattern `project-logs-*`, enabling fast search and filtering.
- **Kibana**: Provides dashboards and visualizations for monitoring pipeline health and microservice activity.

This integration ensures that logs from all services are centralized, searchable, and visualized in real time.

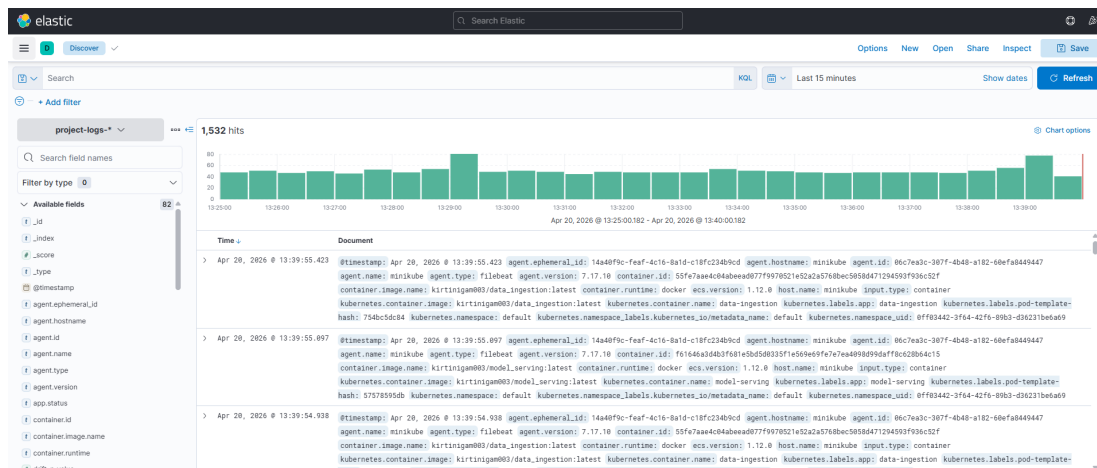


Figure 7.1: ELK stack integration for observability.

## 7.3 Dedicated Dashboards

To improve monitoring granularity, five dashboards were created in Kibana:

- **Data Ingestion Dashboard:** Tracks ingestion requests, schema validation errors, and throughput.
- **Model Training Dashboard:** Visualizes training events, artifact generation, and retraining triggers.
- **Model Serving Dashboard:** Monitors prediction requests, latency, and error rates.
- **Drift Detection Dashboard:** Displays drift flags, per-feature drift statistics, and retraining outcomes.
- **Drift Detection Overview Dashboard:** Aggregates logs across all services, providing a holistic view of pipeline health.

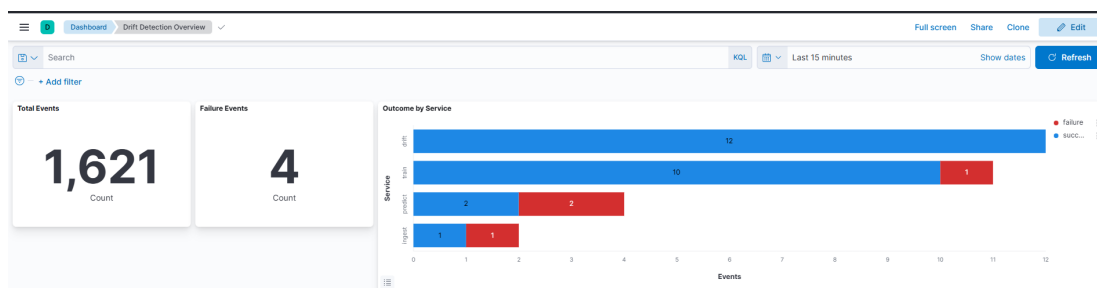


Figure 7.2: Unified Kibana dashboard aggregating logs from all microservices.

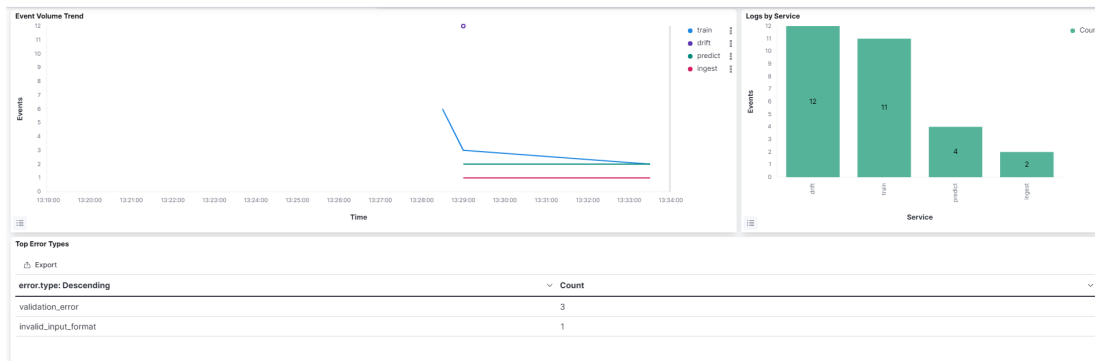


Figure 7.3: Unified Kibana dashboard aggregating logs from all microservices.

## 7.4 Monitoring and Troubleshooting

Monitoring focuses on detecting anomalies and failures early:

- **Failed ingestion events:** Schema mismatches or invalid payloads are flagged.
- **Drift detection anomalies:** Unexpected drift spikes trigger alerts and retraining.
- **Training errors:** Missing artifacts or failed retraining jobs are logged and visualized.
- **Serving latency:** Dashboards highlight prediction delays or error rates.

Terminal logs collected via `kubectl logs` provide raw evidence of failures, which are cross-referenced with Kibana dashboards for root cause analysis.

## 7.5 Operational Scenarios

Operational scenarios demonstrate how observability supports decision-making:

- Heavy drift batch triggers retraining; dashboards confirm retraining completion and new model artifact stored on PVC.
- Serving latency spike detected; ingestion dashboard shows backlog, prompting scaling via HPA.
- Training service crash; pod logs highlight missing dataset, unified dashboard confirms downstream impact.

```
kirti@LAPTOP-8B1OT5K0: $ kubectl get hpa -w
NAME                                REFERENCE                                TARGETS                                MINPODS  MAXPODS  REPLICAS  AGE
data-ingestion-hpa                  Deployment/data-ingestion                 memory: 46%/70%                        1         4         2         6d18h
drift-detection-hpa                  Deployment/drift-detection                 cpu: 2%/80%                             1         3         1         6d18h
model-serving-hpa                    Deployment/model-serving                  cpu: 2%/60%                             2        10         2         6d18h
data-ingestion-hpa                  Deployment/data-ingestion                 memory: 46%/70%                        1         4         2         6d18h
model-serving-hpa                    Deployment/model-serving                  cpu: 1%/60%                             2        10         2         6d18h
data-ingestion-hpa                  Deployment/data-ingestion                 memory: 46%/70%                        1         4         2         6d18h
model-serving-hpa                    Deployment/model-serving                  cpu: 164%/60%                          2        10         2         6d18h
model-serving-hpa                    Deployment/model-serving                  cpu: 164%/60%                          2        10         4         6d18h
model-serving-hpa                    Deployment/model-serving                  cpu: 164%/60%                          2        10         6         6d18h
```

Figure 7.4: Unified Kibana dashboard aggregating logs from all microservices.

# Chapter 8: Root Cause Analysis (RCA) and XAI Diagnostics

## 8.1 Overview: What is happening?

The Root Cause Analysis (RCA) service is a specialized diagnostic layer that extends the platform's drift detection capabilities. While the drift service identifies *that* a statistical shift has occurred in the incoming data, the RCA service investigates *why* the model's behavior might be changing in response to that drift. It acts as a bridge between raw statistical anomalies and actionable model insights.

The service utilizes Explainable AI (XAI) techniques to perform a comparative analysis of feature influence. By evaluating how the model weighs specific features on the baseline training data versus the drifted production samples, it isolates "rogue features" that are disproportionately driving the model's decision-making logic in the new environment.

## 8.2 Motivation: Why are we implementing it?

In production MLOps, drift detection alone can be noisy. Not all data shifts are harmful; some may be benign changes in user demographics that don't affect model accuracy. Implementing RCA provides several critical benefits:

- **Causal Attribution:** It moves beyond simple correlation to identify which specific features are responsible for the model's shifted output.
- **Risk Mitigation:** By identifying "rogue features," engineers can determine if the model is relying on unstable or biased attributes that have drifted.
- **Operational Efficiency:** Instead of blind retraining, RCA allows for targeted debugging, such as identifying a broken data upstream or a specific feature that needs transformation.

## 8.3 The XAI Diagnostic Dashboard

The platform includes a dedicated **AIA Diagnostic Command Center**, a high-performance web dashboard that visualizes the RCA results in real-time.

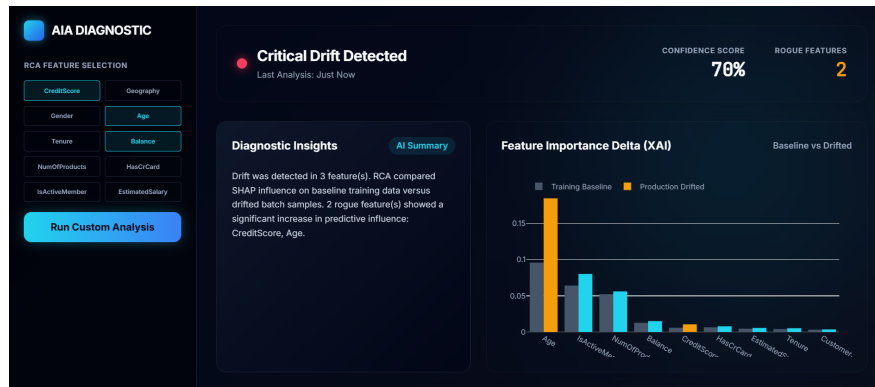


Figure 8.1: RCA Dashboard

### 8.3.1 Dashboard Layout and Navigation

The dashboard is designed for high-density information display, featuring:

- **Executive Header:** Displays the overall model health, confidence scores, and a count of currently identified rogue features.
- **Feature Health Heatmap:** A grid-based visualization of all features, color-coded by their current status (Healthy vs. Rogue).
- **Interactive Control Sidebar:** Allows operators to select specific feature subsets for deep-dive diagnostic runs.

### 8.3.2 Graphs and Visualizations

Two primary charts provide the core diagnostic evidence:

1. **Feature Importance Delta (XAI):** A side-by-side bar chart comparing the "Mean Absolute SHAP" values of the training baseline against the drifted production batch. This chart highlights features that have seen a significant spike in predictive importance (Rogue Features).

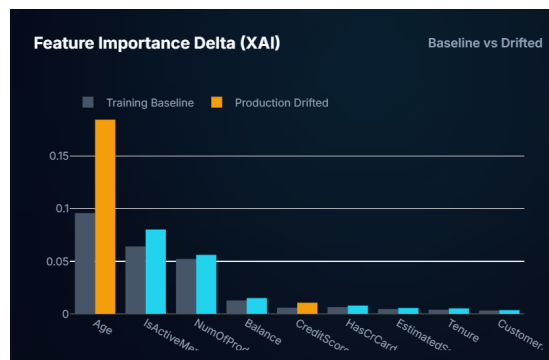


Figure 8.2: Feature Importance

2. **The 'Mirror' Distribution (Statistical Drift):** A dual-histogram view that overlays the baseline distribution with the current production distribution for any selected feature. This allows for visual confirmation of the statistical shift (e.g., mean shift or variance change).

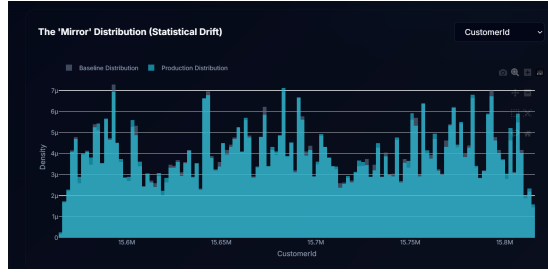


Figure 8.3: Mirror Distribution

### 8.3.3 Interactive Filters

To facilitate efficient debugging, the dashboard provides:

- **Feature Selection List:** Checkboxes to toggle features in and out of the RCA analysis.
- **Contextual Selection:** Clicking any cell in the heatmap automatically updates the distribution histograms to show that feature's data.
- **Real-time RCA Trigger:** A "Run Custom Analysis" button that invokes the backend service with the selected parameters.

## 8.4 Innovation: Why is this innovative?

The integration of RCA into the churn pipeline represents a significant innovation over traditional monitoring systems:

- **Explainable MLOps (X-MLOps):** It shifts the paradigm from "Black Box Monitoring" to "Transparent Diagnostics," making model failures interpretable by human operators.
- **Hybrid Analysis:** By combining statistical tests (KS-style) with attribution-based tests (SHAP), the system ensures that it only alerts on shifts that have a *functional impact* on the model's predictions.
- **Automated Insights:** The service generates "Plain English Explanations" of complex XAI results, lowering the barrier for non-data scientists to understand model health.



# Chapter 9: Security and Governance

## 9.1 Secrets Management

Sensitive variables such as database credentials, API keys, and service tokens were managed using **Ansible Vault**. This ensured that secrets were encrypted at rest and only accessible during pipeline execution. Vault integration allowed Jenkins to inject credentials securely into Ansible playbooks without exposing them in plain text. This practice reduced the risk of accidental leaks in version control and maintained compliance with secure DevOps workflows.

## 9.2 Network and Access

Although the project primarily ran on a local Minikube cluster, network boundaries were enforced by restricting services to **ClusterIP** exposure. Internal URLs were not exposed externally, which prevented unauthorized access from outside the cluster. In a production setting, this would typically be extended with role-based access control (RBAC), TLS certificates, and ingress controllers. While these were not fully implemented here, the design acknowledges the importance of minimizing attack surfaces and limiting external exposure.

## 9.3 Data and Model Lifecycle

Governance policies define how data and models should be managed across their lifecycle. In this project, retraining was triggered automatically by drift detection, and artifacts were stored on a shared PVC. Although formal governance policies (e.g., role-based permissions for triggering retraining, audit trails for artifact integrity) were not fully implemented, the pipeline design anticipates these needs. In a production environment, governance would ensure:

- Clear definition of who can initiate retraining jobs.
- Verification of model artifacts before deployment.
- Versioning and archival of models to maintain reproducibility.

This provides a foundation for extending the project into a more controlled and compliant environment.

# Chapter 10: Results and Evaluation

## 10.1 Functional Validation

Functional validation was performed to ensure that the pipeline behaved correctly under different drift scenarios:

- **No drift scenario:** Ingested data matched the reference distribution, and the drift detection service returned `drift_detected = false`. No retraining was triggered, confirming stability under normal operating conditions.
- **Heavy drift scenario with retraining:** A batch of data with significant distributional changes was ingested. The drift detection service flagged drift, and retraining was triggered automatically. The new model artifact was stored on the PVC, replacing the previous version. Logs confirmed successful retraining and redeployment.

These scenarios validated both the correctness of drift detection and the robustness of retraining integration.

## 10.2 End-to-End Behavior

The end-to-end pipeline was tested by simulating ingestion, prediction, drift detection, and retraining.

- The ingestion path correctly reflected the `drift_response.drift_detected` flag.
- Retraining was triggered when applicable, and the serving service began using the updated model artifact.
- Logs collected from pods confirmed that each microservice executed its responsibilities consistently.

This demonstrated that the microservice architecture, CI/CD pipeline, and observability stack worked together seamlessly.

## 10.3 Limitations

While the system achieved its functional goals, several limitations were observed:

- **Synthetic reference sampling variability:** The reference distribution was generated from synthetic samples, which may not fully capture real-world drift patterns.
- **Scope limited to KS-style drift tests:** Only Kolmogorov–Smirnov style tests were implemented. More advanced drift detection methods (e.g., population stability index, adversarial validation) were not included.
- **Resource constraints:** Running on Minikube with limited memory restricted scalability and performance testing.
- **Governance gaps:** Formal policies for artifact versioning and retraining permissions were not enforced.

# Chapter 11: Visual Appendix

This appendix consolidates all visual evidence from the project, including logs, dashboards, and pipeline execution summaries. It serves as a reference point for readers to connect textual descriptions with actual outputs.

## 11.1 Pipeline Execution

Screenshots of Jenkins pipeline runs illustrate the execution times and outcomes of each stage.

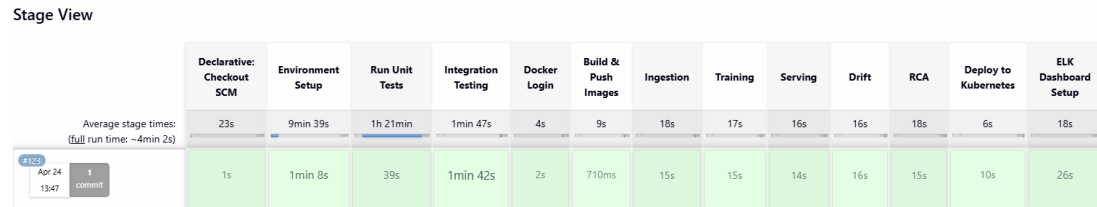


Figure 11.1: Jenkins pipeline execution summary with stage timings.

## 11.2 Pod Logs

Terminal logs collected via `kubectl logs` provide raw diagnostic evidence of microservice behavior.

```
10.244.0.1 - - [20/Apr/2026 07:59:11] "GET /health HTTP/1.1" 200 -
10.244.0.1 - - [20/Apr/2026 07:59:13] "GET /health HTTP/1.1" 200 -
10.244.0.1 - - [20/Apr/2026 07:59:16] "GET /health HTTP/1.1" 200 -
{"service": {"name": "ingest"}, "event": {"outcome": "success", "type": "request"}, "app": {"status": "success"}, "rows": 2, "duration_ms": 1235, "ingest": {"source": "unknown"}, "dependencies": {"serving_duration_ms": 64, "drift_duration_ms": 1111}, "http": {"request": {"method": "POST", "path": "/ingest"}}}
127.0.0.1 - - [20/Apr/2026 07:59:18] "POST /ingest HTTP/1.1" 200 -
10.244.0.1 - - [20/Apr/2026 07:59:21] "GET /health HTTP/1.1" 200 -
```

Figure 11.2: Logs from ingestion pod

```
{"service": {"name": "train"}, "event": {"outcome": "success", "type": "training_run_completed"}, "app": {"status": "success"}, "training_run_id": "a5f02b4f-4f97-4167-98b7-5816c925791e", "training_completed_at": "2026-04-20T07:59:18.550735+00:00", "duration_ms": 1033, "training_duration_ms": 1033, "validation_accuracy": 1.0, "model_version": "v1.0", "resource_summary": {"max_rss_bytes": 174325760, "avg_rss_bytes": 174325760.0, "max_cpu_percent": 0.0, "avg_cpu_percent": 0.0}}
10.244.1.210 - - [20/Apr/2026 07:59:18] "POST /train HTTP/1.1" 200 -
10.244.0.1 - - [20/Apr/2026 07:59:19] "GET /health HTTP/1.1" 200 -
```

Figure 11.3: Logs from training pod

```
{"service": {"name": "drift"}, "event": {"outcome": "success", "type": "feature_drift"}, "app": {"status": "success"}, "drift": {"feature": "EstimatedSalary", "drifted": false, "p_value": 1.0, "confidence": 0.0, "severity": "low", "reference_mean": 112574.82273434385, "reference_std": 50292.865584904204, "live_mean": 49503.5, "live_std": NaN, "duration_ms": 117, "http": {"request": {"method": "POST", "path": "/drift"}}}, "drift_p_value": 1.0}
127.0.0.1 - - [20/Apr/2026 07:59:01] "POST /drift HTTP/1.1" 200 -
10.244.0.1 - - [20/Apr/2026 07:59:03] "GET /health HTTP/1.1" 200 -
10.244.0.1 - - [20/Apr/2026 07:59:03] "GET /health HTTP/1.1" 200 -
```

Figure 11.4: Logs from drift pod

```

10.244.0.1 - - [20/Apr/2026 07:59:10] "GET /health HTTP/1.1" 200 -
10.244.0.1 - - [20/Apr/2026 07:59:15] "GET /health HTTP/1.1" 200 -
{"service": {"name": "predict"}, "event": {"outcome": "success", "type": "request"}, "app": {"status": "success", "customer_id": 123, "duration_ms": 35, "client": {"source": "unknown"}, "http": {"request": {"method": "POST", "path": "/predict"}}, "prediction": {"prediction": [1, 1], "churn_probability": [0.9949134411077678, 0.9984707442492561]}}
10.244.1.199 - - [20/Apr/2026 07:59:17] "POST /predict HTTP/1.1" 200 -

```

Figure 11.5: Logs from serving pod

## 11.3 Kibana Dashboards

Dedicated dashboards provide observability for each microservice and a unified view of pipeline health.



Figure 11.6: Kibana Dashboard for Data Ingestion

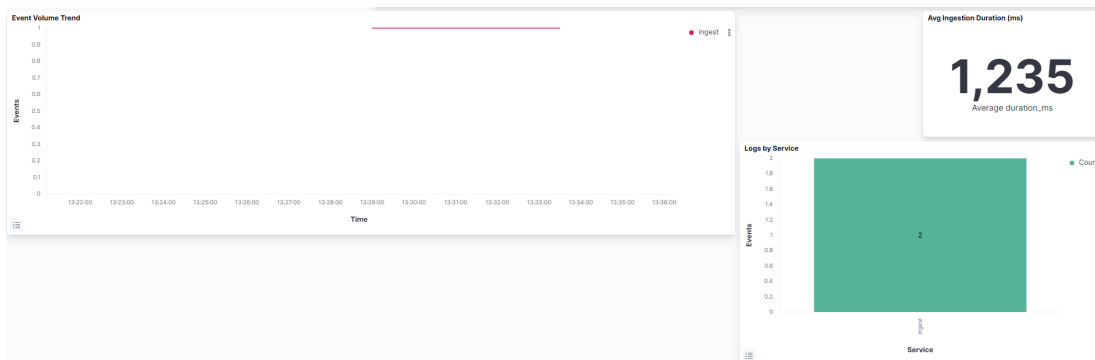


Figure 11.7: Kibana Dashboard for Data Ingestion



Figure 11.8: Kibana Dashboard for Model Training

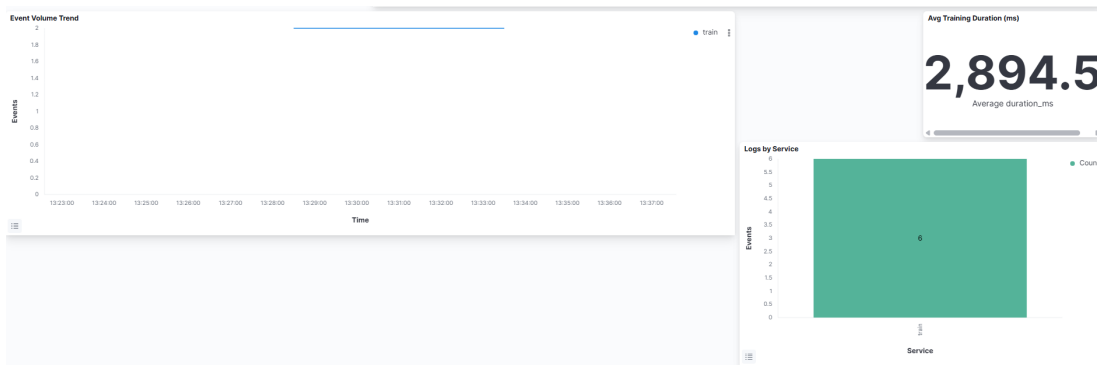


Figure 11.9: Kibana dashboard for Model Training



Figure 11.10: Kibana dashboard for Model Serving

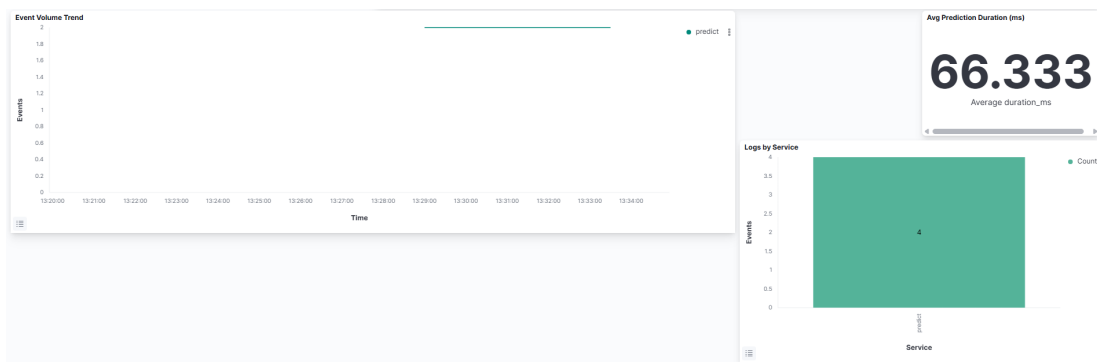


Figure 11.11: Kibana dashboard for Model Serving



Figure 11.12: Kibana dashboard for Drift Detection

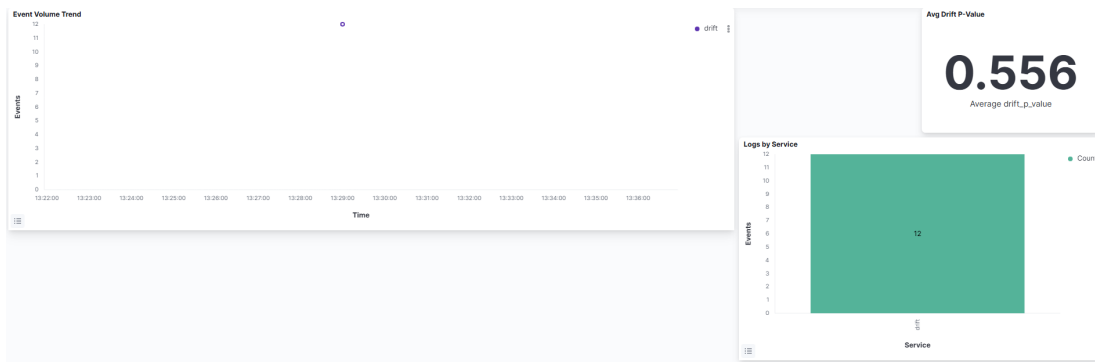


Figure 11.13: Kibana dashboard for Drift Detection

## 11.4 Operational Scenarios

Visualizations of operational scenarios demonstrate how the system responded to drift and retraining events.

```
kirti@LAPTOP-8B10T5K0:~$ export BASELINE='[{"CustomerId":1,"CreditScore":600,"Geography":"France","Gender":"Male","Age":30,"Tenure":5,"Balance":1800,"NumOfProducts":1,"HasCrCard":1,"IsActiveMember":1,"EstimatedSalary":40000,"Exited":0}, {"CustomerId":2,"CreditScore":650,"Geography":"Germany","Gender":"Female","Age":45,"Tenure":1,"Balance":5000,"NumOfProducts":2,"HasCrCard":0,"IsActiveMember":0,"EstimatedSalary":50000,"Exited":1}]'
```

Figure 11.14: Train once with a fixed reference (inline JSON is easiest so you know what the reference looks like)

```
kirti@LAPTOP-8B10T5K0:~$ export NO_DRIFT="$BASELINE"
curl -sS -X POST "$DRIFT/drift" -H "Content-Type: application/json" -d "$NO_DRIFT" | tee /tmp/drift-no.json
# Expect: "drift_detected": false
grep -o '"drift_detected": [^,]*' /tmp/drift-no.json
{"details":{"Age_drift":false,"Balance_drift":false,"CreditScore_drift":false,"CustomerId_drift":false,"EstimatedSalary_drift":false,"HasCrCard_drift":false,"IsActiveMember_drift":false,"NumOfProducts_drift":false,"Tenure_drift":false,"concept_drift":"monitor prediction accuracy over time","label_drift":false},"drift_detected":false}
```

Figure 11.15: Re-send the same distribution you trained on (same two rows as BASELINE). Feature and label summaries stay aligned with the saved reference, so drift is usually not flagged

```
kirti@LAPTOP-8B10T5K0:~$ export DRIFT_HEAVY='[{"CustomerId":901,"CreditScore":300,"Geography":"France","Gender":"Male","Age":95,"Tenure":50,"Balance":999999,"NumOfProducts":1,"HasCrCard":0,"IsActiveMember":0,"EstimatedSalary":200000,"Exited":1}, {"CustomerId":902,"CreditScore":250,"Geography":"France","Gender":"Male","Age":99,"Tenure":40,"Balance":888888,"NumOfProducts":1,"HasCrCard":0,"IsActiveMember":0,"EstimatedSalary":250000,"Exited":1}, {"CustomerId":903,"CreditScore":700,"Geography":"Germany","Gender":"Female","Age":45,"Tenure":10,"Balance":5000,"NumOfProducts":2,"HasCrCard":1,"IsActiveMember":1,"EstimatedSalary":60000,"Exited":0}]'
```

Figure 11.16: Rows far outside the training distribution (extreme ages, balances, credit scores, and skewed labels). This pattern normally triggers drift and re-training when TRAINING\_URL is set.

```
kirti@LAPTOP-8B10T5K0:~$ kubectl get hpa -w
```

| NAME                | REFERENCE                  | TARGETS         | MINPODS | MAXPODS | REPLICAS | AGE   |
|---------------------|----------------------------|-----------------|---------|---------|----------|-------|
| data-ingestion-hpa  | Deployment/data-ingestion  | memory: 46%/70% | 1       | 4       | 2        | 6d18h |
| drift-detection-hpa | Deployment/drift-detection | cpu: 2%/80%     | 1       | 3       | 1        | 6d18h |
| model-serving-hpa   | Deployment/model-serving   | cpu: 2%/60%     | 2       | 10      | 2        | 6d18h |
| data-ingestion-hpa  | Deployment/data-ingestion  | memory: 46%/70% | 1       | 4       | 2        | 6d18h |
| model-serving-hpa   | Deployment/model-serving   | cpu: 1%/60%     | 2       | 10      | 2        | 6d18h |
| data-ingestion-hpa  | Deployment/data-ingestion  | memory: 46%/70% | 1       | 4       | 2        | 6d18h |
| model-serving-hpa   | Deployment/model-serving   | cpu: 164%/60%   | 2       | 10      | 2        | 6d18h |
| model-serving-hpa   | Deployment/model-serving   | cpu: 164%/60%   | 2       | 10      | 4        | 6d18h |
| model-serving-hpa   | Deployment/model-serving   | cpu: 164%/60%   | 2       | 10      | 6        | 6d18h |

Figure 11.17: Operational scenario: serving latency spike detected and mitigated via HPA scaling.

# Chapter 12: Conclusion and Future Work

## 12.1 Summary

The project successfully delivered a production-grade MLOps pipeline built on microservices, containerization, and Kubernetes orchestration. Key achievements included:

- Automated CI/CD pipeline using Jenkins, ensuring reproducible builds and deployments.
- Observability through structured logging and ELK stack dashboards, enabling real-time monitoring.
- Drift detection integrated with retraining, ensuring models remain trustworthy over time.
- Secure secrets management with Ansible Vault, reducing risks of credential exposure.

Together, these components demonstrated how operational discipline can transform a machine learning prototype into a robust production system.

## 12.2 Lessons Learned

Several lessons emerged during the project:

- **Balancing complexity and simplicity:** While Kubernetes and ELK stack added operational robustness, they also introduced resource overheads in Minikube. This highlighted the trade-off between platform sophistication and lightweight ML workflows.
- **Importance of observability:** Dashboards and logs proved invaluable for diagnosing drift detection errors and retraining failures, reinforcing the need for structured monitoring in production ML.
- **CI/CD discipline:** Automating builds and deployments reduced manual errors and ensured consistency, but required careful pipeline design to handle failures gracefully.
- **Data quality dependency:** Retraining failures due to single-class batches emphasized that operational pipelines are only as strong as the data they process.



## 12.3 Future Improvements

Potential enhancements to strengthen the pipeline include:

- **Stronger monitoring and alerting:** Integrating alerting systems (e.g., Prometheus + Alertmanager) to notify operators of drift spikes, failed re-training, or serving latency anomalies.
- **A/B testing for model evaluation:** Deploying multiple model versions in parallel to compare performance on live traffic, ensuring retrained models outperform baselines before full rollout.
- **Feature store integration:** Centralizing feature engineering and storage to improve consistency across training and serving, reducing data leakage risks.
- **Governance and compliance:** Adding role-based access control (RBAC) for retraining triggers, artifact versioning, and audit trails to align with enterprise ML governance standards.
- **Scalability:** Migrating from Minikube to a multi-node Kubernetes cluster or cloud-native environment to support larger workloads and higher availability.

# Chapter 13: References

For more details on the project, please refer to the [GitHub Repository](#).

For more details on the Docker Hub Image, please refer to the [Docker Hub Image](#).